Loss Run Intelligence — why each Azure service is in the solution

One row per service: what it does in this proof of concept, why it is required rather than optional, and what was considered instead. The architecture diagram (separate PDF) shows how they connect; the services table shows the POC deployment against the recommended organisation setup.

Application and web

Service	What it does here	Why it is required	Without it / alternative considered
Azure Functions (Flex Consumption) with Durable Functions	Runs the whole pipeline: accept the upload, classify what is in it, recognise the carrier format, replay or learn it, validate against printed totals, write results, serve the API, chat and integration.	A loss run can take seconds (replay) or minutes (learn, then a human review that may wait days). Durable orchestrations keep that state per document and resume after a wait; consumption billing means idle time costs nothing between submissions.	A plain web API would need its own job queue, retry logic and state store to survive a human-review wait; the alternative (Container Apps + a queue) is more to run with no benefit at POC volume.
Durable Task Scheduler	The managed store behind Durable Functions: every orchestration's history, timers and pending human-review events.	It is what lets a document sit in review for days and resume exactly where it stopped, and what makes a crashed worker harmless (the next worker replays the history). It was the largest non-AI line in the bill, so it is included in the per-document cost.	The older storage-account backend works but is slower under concurrency and has no dashboard; the scheduler is Microsoft's current recommendation.
Static Web Apps	Hosts the React workbench (upload, review queue, lineage trace, analytics, what-if, chat, costs) on a global edge with a managed certificate; also hosts the offline demo site.	The front end is static files; there is no server to patch or scale. The offline mode means the demo can be shown with no backend at all, and the custom domain keeps Azure resource names out of what stakeholders see.	App Service would work but is a server to run for files that never change; a storage-account website has no managed certificate or custom-domain validation.

AI services

Service	What it does here	Why it is required	Without it / alternative considered
Microsoft Foundry · Content Understanding	Reads PDFs and scans once: page layout, tables, which document is which inside a bundle (loss run vs payroll report vs application), and per-field confidence with page coordinates.	It is the only Azure service that gives layout + classification + confidence + coordinates in one call with no labelled training data. The coordinates are what make cell-level lineage possible; the confidence is what gates straight-through vs review.	Document Intelligence alone gives layout but not classification or bundle splitting (it was measured as a second opinion: no accuracy gain, so it is off). A general model reading the page gives no coordinates and no confidence.
Language models (gpt-5.4-mini, Data Zone US)	Used only when a carrier format is new or has drifted: the model proposes a format specification, the system verifies it against the printed totals, and from then on the format is replayed with no model call.	Without a model, every new carrier format would need a developer to hand-write a mapping. With a model on every document, cost scales with volume and answers can vary. Learning once gives both: 47 % of corpus documents needed no model call, and a format costs about $0.05 to learn.	Calling a model per document (what most vendors do) was measured: format memory breaks even by the 3rd or 4th document of a carrier and is deterministic afterwards. Data Zone keeps processing in the US.
Embeddings (text-embedding-3-large)	Turns a document's header region into a vector so a carrier format can be recognised even when the file name and layout details differ; also indexes pages for chat.	Format recognition by exact header hash alone misses reprints, re-exports and scans. The embedding is the second tier of the fingerprint (cosine 1.0 for the same format vs 0.64 for a payroll report) and costs fractions of a cent.	Keyword matching was tried first and confused loss runs with payroll reports from the same carrier.

Data

Service	What it does here	Why it is required	Without it / alternative considered
Azure SQL (ledger and temporal tables)	System of record: claims, every extracted value with its page/cell origin, every human change, feature snapshots for analytics, chat turns, cost events.	Two requirement items are only satisfiable here: cell-level lineage (a relational model joining value → cell → page → document → format version) and a tamper-evident audit (SQL ledger tables are append-only and cryptographically chained; temporal tables keep every prior state). Underwriters also query it directly for analytics at p95 158 ms.	Cosmos DB has no ledger feature and no temporal history; a document store would need the audit chain built by hand, which is exactly what an auditor would then have to trust.
Blob Storage	Keeps the original uploads immutable (the evidence), the raw extraction output, the learned format registry, exports, ledger digests and the daily cost export.	The original file must be preserved unchanged for the lineage to mean anything; immutability policies make that a platform guarantee rather than a promise. Formats are versioned files so a learned format can be inspected, retired and rolled back.	Storing documents in SQL bloats the database and loses the immutability policy; a file share has no versioning or retention lock.
Key Vault	Holds the few secrets that exist (SQL admin, webhook signing key) and nothing else; every service-to-service call uses managed identity.	No key in application settings means nothing to leak from a config export and nothing to rotate by hand. It is a one-line answer in the security review.	App settings with keys are the default and the thing security reviews reject first.

Integration and search

Service	What it does here	Why it is required	Without it / alternative considered
Azure AI Search	Index over document pages, sheets and claims so the chat can answer a question scoped to one quote, a workspace or all history, with a citation to the page or cell.	Chat without an index would mean sending whole documents to the model every turn (slow, costly, and no citation). Scope is enforced at the search layer, which is why the 11-case leakage test shows 0 leaks.	Searching SQL with LIKE does not rank, does not understand synonyms and cannot cite a page region; it was not viable for the historical-archive scenario.
Service Bus	Reliable hand-off of results (flags, annotations, memos) to the quote-management system through a transactional outbox with idempotent commands and a dead-letter queue.	The quote system is a separate system that may be down or slow. The outbox guarantees a result is sent exactly once, in order, and that a failure is visible rather than silent. Load test: 0 dead-lettered messages.	A direct HTTP call from the pipeline loses results when the target is unavailable and can send duplicates on retry; Storage Queues have no sessions or duplicate detection.
Event Grid	Publishes events (upload completed, review decided) so other systems can react without polling.	Lets the quote system, a dashboard or a future notification service subscribe to what happened without the pipeline knowing who is listening.	Polling the API from every interested system; or coupling the pipeline to each consumer.

Platform, identity, security

Service	What it does here	Why it is required	Without it / alternative considered
Microsoft Entra ID	Sign-in and role-based access: Applicant, Underwriter, UW Manager, Admin as app roles; managed identities for every service-to-service hop.	Who overrode a value is only meaningful if identity is real. App roles drive what each screen allows; managed identities remove passwords between services. It is the organisation's identity provider already, so production is a tenant change, not a code change.	Local accounts would need their own password store and would not satisfy the organisation's access policy.
Application Insights · Log Analytics	Telemetry for every request and orchestration step: latency percentiles, errors, model token counts, cost per document; the evidence behind the acceptance criteria.	Every number quoted in the acceptance table (p95 latency, model calls per document, cost per path) comes from here or from the metered events it correlates with. Without it the claims are anecdotes.	Console logs; no percentiles, no correlation across the pipeline, no cost query.
Azure Load Testing	Runs the 25-concurrent-underwriter scenario against the live system and records p95 per operation.	The requirement is a concurrency target, so it had to be measured rather than estimated: override p95 860 ms, features p95 158 ms, chat p50 2.9 s, 0 errors. It also found and verified the fix for the earlier 1.87 s override path.	A local load tool from one laptop measures the laptop's network, not the system.
Defender for Cloud (AI threat protection)	Monitors the AI endpoints and the rest of the environment for prompt-injection patterns, data exfiltration attempts and misconfiguration.	The chat accepts free text from users over claim data; threat protection is the detective control, and its alerts route to the security team in the organisation's subscription.	No monitoring of the AI surface; the governance pack would have a gap.
Terraform (infrastructure as code)	Every resource above is declared in Terraform with remote state; there are no portal-made resources.	It is what makes the handover real: the organisation's subscription is a `terraform apply` with different variables, not a rebuild. Any manual change is captured back or recorded in a decision record.	Portal-built environments cannot be reproduced or reviewed; the POC would not transfer.

Cross-cutting, built into every step: every billable call is metered to the document it served and reconciled to the Azure bill; every extracted value keeps its page-and-cell origin, format version and confidence; every change is an append-only audit event with user, role, time, old → new and reason.